FEDFUI

WEEK-1

Experiment Title	Analyze and implement efficient UI updates by comparing Direct DOM vs Virtual DOM
Subject	Front-End Engineering & Framework Design
Tools Used	VS Code, HTML5, JavaScript (ES6), Browser DevTools

1 Aim & Problem Description

1.1 Aim

To analyze and implement efficient UI updates by understanding DOM manipulation, Virtual DOM, and the browser rendering pipeline. Students will compare the performance of traditional direct DOM updates against a simulated Virtual DOM approach, measure timing using browser APIs, and draw engineering conclusions about why modern front-end frameworks adopt the Virtual DOM abstraction.

1.2 Problem Description

A college attendance management system displays a large number of student records and updates their attendance status dynamically during class hours. The system is built using plain JavaScript with direct DOM manipulation. As the number of students grows and attendance updates become more frequent, the system begins to slow down significantly. The core problem is that even when only a few students change their attendance status, the system rebuilds the entire student list from scratch. Every rebuild triggers the browser to go through its full rendering pipeline — constructing the DOM tree, computing styles, performing layout (reflow), and then painting every pixel back to the screen (repaint). This is enormously wasteful when only 10 out of 500 records actually changed.

The experiment simulates this real-world problem and demonstrates how a Virtual DOM approach — the technique at the core of React, Vue, and Preact solves it by identifying only the changed records and applying surgical, minimal updates to the real DOM.

> **Core Engineering Problem**
>
> Unnecessary full re-renders triggered by direct DOM manipulation cause reflows and repaints for every update, regardless of how small the change is. This breaks the performance contract: small data change should produce small UI cost.

1.3 Modules Covered

- **DOM Manipulation using JavaScript**
- Virtual DOM Simulation using plain JavaScript objects
- Browser Rendering Pipeline — Reflow and Repaint
- Performance Comparison using console.time()

1.4 Tasks

1. Create a webpage displaying a large student list (500 records)
2. Implement direct DOM manipulation to update all records
3. Simulate a Virtual DOM using JavaScript objects
4. Implement a diffing algorithm to detect only changed elements
5. Apply minimal patches to the real DOM using the changeset
6. Measure and compare performance of both approaches

2 Theory & Concept Explanation

2.1 What is the DOM?

The Document Object Model (DOM) is a programming interface for HTML and XML documents. It represents the entire web page as a structured tree of objects, where each HTML element becomes a node in the tree. The browser constructs this tree when it first loads the HTML and keeps it in memory as a live, interactive data structure throughout the life of the page.

Developers can interact with the DOM using JavaScript — reading element properties, changing text content, adding or removing nodes, and modifying CSS classes. Every interaction with the DOM, however, comes with a cost. The browser must synchronise its internal state every time you touch the DOM, which can be computationally expensive.

The DOM is not just a passive data structure. It is deeply connected to the browser's rendering engine. Any property that affects the visual layout of the page — like width, height, font size, or position — forces the browser to recalculate the geometry of the entire page before it can show you the result. This process is called reflow.

> **Key Definition — DOM**
> The DOM is a live, mutable tree of objects representing the HTML document. It is the browser's authoritative record of what is on the page. Touching it triggers synchronisation with the rendering engine, which can be expensive if done carelessly.

2.2 The Browser Rendering Pipeline

When a browser receives an HTML page, it does not draw pixels immediately. It goes through a structured pipeline of stages, each feeding into the next. Understanding this pipeline is essential to understanding why DOM manipulation has performance costs.

Stage 1 HTML Parsing and DOM Construction

The browser reads the raw HTML text and converts it into a tree of node objects. This tree is the DOM. Each HTML tag becomes an element node, each text content becomes a text node, and attributes become properties of those nodes. The browser parses top-to-bottom; if it encounters a script tag, it pauses parsing to execute the script before continuing. This is why JavaScript that manipulates the DOM should be placed at the bottom of the HTML body or deferred.

Stage 2 — CSS Parsing and CSSOM Construction

Simultaneously with HTML parsing, the browser parses all CSS — from style tags, linked stylesheets, and inline styles — and builds the CSS Object Model (CSSOM). The CSSOM is another tree structure that maps selectors to computed style rules. Every element in the DOM will eventually be matched against the CSSOM to determine its final visual properties.

Stage 3 — Render Tree Construction

The DOM and CSSOM are combined to create the Render Tree. The Render Tree contains only the nodes that are actually visible on the page (elements with display:none are excluded). Each node in the Render Tree carries the computed style information from the CSSOM. This is the tree the browser will use for layout and painting.

Stage 4 — Layout (Reflow)

During layout, the browser calculates the exact position and dimensions of every node in the Render Tree. It determines where each element sits on the page, how wide and tall it is, and how it interacts with neighbouring elements. This process considers the viewport size, CSS box model rules, flexbox and grid constraints, and font metrics. A layout calculation for the full page can be expensive because changing one element can cascade and affect the position of dozens of others.

Reflow is triggered whenever any change affects the geometry of elements — adding or removing nodes, changing font sizes, modifying padding or margins, or changing the content of a text node. Reading certain DOM properties (like offsetWidth, getBoundingClientRect, or scrollTop) after a write operation forces a synchronous reflow, because the browser must give you the up-to-date value. Alternating reads and writes in a loop is the classic cause of layout thrashing.

Stage 5 — Paint (Repaint)

After layout, the browser knows where everything is. The paint stage converts those positions and styles into actual pixel values on a bitmap. This includes drawing text, borders, background colours, shadows, and images. Paint is performed in layers — elements with the z-index property or certain CSS transforms get their own compositing layer, which can be updated independently without repainting other layers.

Stage 6 — Composite

Finally, the browser's compositor takes all the painted layers and combines them in the correct order to produce the final image shown on screen. The compositor communicates with the GPU for hardware-accelerated rendering. CSS properties that are composited (like transform and opacity) can be animated at 60fps without triggering layout or paint, because they are handled entirely by the GPU on a separate thread.

Reflow (Layout)	Repaint (Paint)
Triggered by: geometry changes	Triggered by: visual changes
Examples: width, height, font-size	Examples: color, background, border-color
Most expensive operation	Less expensive than reflow
Invalidates entire layout tree	Invalidates only affected pixels
Always followed by repaint	Does NOT always trigger reflow
Avoid: alternating DOM reads+writes	Avoid: unnecessary style changes

2.3 Why Direct DOM Manipulation is Slow

When developers manipulate the DOM directly — by setting innerHTML, changing textContent, or calling appendChild in a loop — they are working against the rendering pipeline. The problem is not that any single DOM operation is impossibly slow. The problem is that naive code causes many such operations in rapid succession, each triggering reflow and repaint.

Consider the attendance system. When a teacher clicks 'Update', the direct DOM approach clears the entire list (one DOM write that destroys 500 list items) and then recreates all 500 items from scratch (500 more DOM writes). The browser must handle this destruction and reconstruction as a burst of mutations, triggering layout recalculation at least once and repainting the entire visible portion of the list.

If only 50 out of 500 students actually changed their status, then 450 of those writes were completely unnecessary. The browser still had to do all the work. This is the fundamental inefficiency: direct DOM manipulation has no memory of what already exists on screen, so it always assumes everything needs to be replaced.

> **Why It Scales Badly**
> With 500 students, direct DOM update might take 8ms. With 5,000 students, it could take 80ms — dropping below 12fps and making the UI feel unresponsive. The Virtual DOM approach, by contrast, scales with the NUMBER OF CHANGES, not the total list size.

2.4 What is Virtual DOM?

The Virtual DOM is an engineering abstraction — a lightweight JavaScript object tree that represents what the UI should look like. It is not the actual browser DOM. It is simply a plain JavaScript data structure: an array of objects where each object describes one UI element (its type, properties, and text content).

The key insight of the Virtual DOM approach is the separation between description and execution. Instead of telling the browser what to do (imperative: 'change this element'), you describe what the UI should look like (declarative: 'here is the full desired state') and let an algorithm figure out the minimal set of actual DOM mutations required.

When state changes, a new Virtual DOM tree is created representing the desired new UI state. A diffing algorithm then compares the new VDOM tree against the old one, node by node, to find the differences. Only the differences become real DOM mutations. If a node is identical in both trees, the real DOM element is not touched at all.

This means the performance cost of a UI update scales with the number of changes, not the total size of the UI. Updating 10 students out of 500 becomes 10 DOM operations, not 500. The rendering pipeline only fires for those 10 elements, not for the entire list.

Direct DOM Approach	Virtual DOM Approach
Mutates DOM immediately and directly	Describes desired state as JS object
No memory of previous state	Compares old vs new virtual tree
Full re-render on every update	Updates only changed nodes
Cost: O(total elements)	Cost: O(changed elements)
Reflow + repaint for all elements	Reflow + repaint for changed only
Fast for tiny UIs	Scales efficiently for large UIs

2.5 The Diffing Algorithm

The diffing algorithm is the heart of the Virtual DOM system. Its job is to compare two versions of the virtual tree old and new — and produce a minimal list of changes that must be applied to the real DOM. This is sometimes called computing the 'changeset' or 'patch'.

In the simplified implementation used in this experiment, the diffing algorithm works at the level of individual list items. It iterates through every node in the new virtual tree and compares it against the corresponding node in the old virtual tree. If a node's text content differs between old and new, that node is added to the changeset. If they are identical, the node is skipped entirely.

In production frameworks like React, the diffing algorithm is significantly more sophisticated. It handles additions and deletions of nodes, reordering (using the 'key' prop to track element identity across renders), type changes (where an element changes from a to a), and recursive diffing of nested trees. React's algorithm reduces the theoretical $O(n^3)$ complexity of general tree diffing to $O(n)$ by applying two practical heuristics: nodes of different types produce different trees, and stable keys across renders indicate the same logical element.

> **Diffing in a Nutshell**
>
> For each node at position i: if oldVDOM[i].text === newVDOM[i].text, skip it. Otherwise, add {index: i, text: newVDOM[i].text} to the changeset. Apply only the changeset to the real DOM. The real DOM is never touched for unchanged elements.

3 Project Structure & Setup

3.1 Project Folder Structure

The experiment is structured as a self-contained VS Code project with three files. This flat structure keeps everything simple while demonstrating the core concepts clearly.

```
Week1/
│
├── index.html     ← Main page: buttons + student list container
├── style.css      ← Basic visual styling for the list
└── script.js      ← All logic: data generation, DOM update,
                     Virtual DOM creation, diffing, and patching
```

3.2 Tools Required

Editor	Visual Studio Code (VS Code) — free, cross-platform code editor
Language	JavaScript (ES6+) — let, const, arrow functions, spread operator, template literals
Markup	HTML5 — semantic tags, button onclick handlers, ul/li structure
Styling	CSS3 — basic layout and typography, no frameworks needed
DevTools	Chrome or Firefox DevTools — Performance panel, Console for timing output
Run Method	Open index.html directly in browser, or use VS Code Live Server extension

3.3 How to Run the Project

7. Open VS Code and create a new folder named week1
8. Create three files inside: index.html, style.css, and script.js
9. Copy the code from each section below into the respective file
10. Right-click index.html in VS Code and select 'Open with Live Server', or simply open the file in any modern browser
11. Open the browser DevTools console (F12 → Console tab)
12. Click 'Update Using DOM' and observe the timing logged to console
13. Click 'Update Using Virtual DOM' and compare the timing
14. Repeat several times and record average values

4 Complete Code

4.1 index.html — The HTML Page

The HTML file defines the structure of the attendance system. It is intentionally minimal — its job is only to provide the containers and buttons that JavaScript will use. All the logic lives in script.js.

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Attendance System</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="container">

    <h1>Student Attendance System</h1>

    <!-- Form Section -->
    <div class="form">
      <input type="number" id="id" placeholder="Student ID">
      <input type="text" id="name" placeholder="Student Name">

      <select id="status">
        <option value="true">Present</option>
        <option value="false">Absent</option>
      </select>

      <div class="btn-group">
        <button onclick="addStudent()">Add</button>
        <button onclick="updateStudent()">Update</button>
      </div>
    </div>

    <!-- DOM vs VDOM Buttons -->
    <div class="btn-group">
      <button onclick="updateDOM()">Update Using DOM</button>
      <button onclick="updateVDOM()">Update Using VDOM</button>
    </div>

    <!-- Table Section -->
    <div class="table-container">
      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Attendance</th>
            <th>Action</th>
```

```
        </tr>
      </thead>
      <tbody id="tableBody">
        <!-- Data comes from script.js -->
      </tbody>
    </table>
  </div>

</div>
```

4.2 style.css :The Stylesheet

The CSS file provides minimal but intentional styling. The goal is to make the list readable while keeping the code simple enough that the styling itself does not interfere with the performance measurements.

```
/* Center entire content */
body {
  font-family: Arial;
  background: #f4f6f9;
  margin: 0;
  height: 100vh;
  display: flex;
  justify-content: center;   /* horizontal center */
  align-items: center;       /* vertical center */
}

/* Main container */
.container {
  width: 90%;
  max-width: 900px;
  background: white;
  padding: 25px;
  border-radius: 10px;
  box-shadow: 0px 4px 15px rgba(0,0,0,0.2);
  text-align: center;
}

/* Heading */
h1 {
  color: #1a3a5c;
  margin-bottom: 20px;
}

/* Form inputs */
.form input, .form select {
  padding: 8px;
  margin: 5px;
  width: 150px;
}

/* Button group */
.btn-group {
```

```
  margin-top: 10px;
}

/* Buttons */
button {
  padding: 8px 15px;
  margin: 5px;
  background: #1565C0;
  color: white;
  border: none;
  cursor: pointer;
  border-radius: 5px;
}

button:hover {
  background: #0D47A1;
}

/* Table container */
.table-container {
  margin-top: 20px;
  max-height: 300px;
  overflow-y: auto;   /* scroll for large data */
}

/* Table styling */
table {
  width: 100%;
  border-collapse: collapse;
}

th, td {
  padding: 10px;
  text-align: center;
}

th {
  background: #1565C0;
  color: white;
}

tr:nth-child(even) {
  background: #f2f2f2;
}

/* Highlight updates */
.updated {
  background-color: yellow !important;
}
```

4.3 script.js : Complete Logic

The script file contains all application logic. It is organised into five distinct parts: data generation, initial DOM rendering, direct DOM update, Virtual DOM simulation, and the Virtual DOM update function. Each part is explained in detail below.

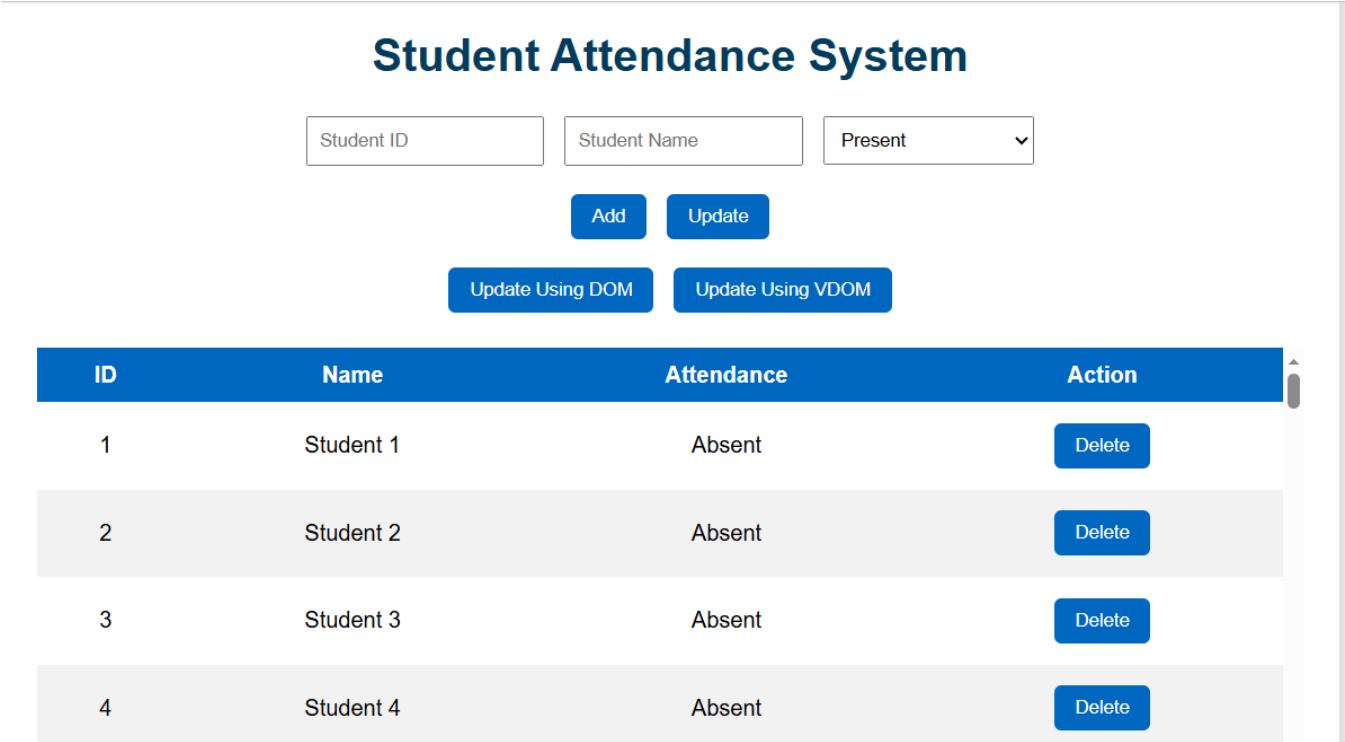
```
let students = [];

// ------------------------------
// INITIAL DATA
// ------------------------------
for (let i = 1; i <= 50; i++) {
  students.push({
    id: i,
    name: "Student " + i,
    present: false
  });
}

// ------------------------------
// RENDER TABLE
// ------------------------------
function render(data) {
  const table = document.getElementById("tableBody");
  table.innerHTML = "";

  data.forEach((s, index) => {
    const row = document.createElement("tr");

    row.innerHTML = `
      <td>${s.id}</td>
      <td>${s.name}</td>
      <td>${s.present ? "Present" : "Absent"}</td>
      <td>
        <button onclick="deleteStudent(${index})">Delete</button>
      </td>
    `;

    table.appendChild(row);
  });
}

window.onload = () => render(students);

// ------------------------------
// ADD STUDENT
// ------------------------------
function addStudent() {
  const id = document.getElementById("id").value;
  const name = document.getElementById("name").value;
  const present = document.getElementById("status").value === "true";

  students.push({ id, name, present });
```

```
  render(students);
}

// ----------------------------
// UPDATE STUDENT
// ----------------------------
function updateStudent() {
  const id = document.getElementById("id").value;
  const name = document.getElementById("name").value;
  const present = document.getElementById("status").value === "true";

  const index = students.findIndex(s => s.id == id);

  if (index !== -1) {
    students[index] = { id, name, present };

    render(students);

    // Highlight updated row
    setTimeout(() => {
      document.getElementById("tableBody").rows[index].classList.add("updated");
    }, 50);
  } else {
    alert("Student not found!");
  }
}

// ----------------------------
// DELETE STUDENT
// ----------------------------
function deleteStudent(index) {
  students.splice(index, 1);
  render(students);
}

// ----------------------------
// DOM UPDATE (Slow)
// ----------------------------
function updateDOM() {

  console.time("DOM Update");

  const table = document.getElementById("tableBody");

  students.forEach((s, index) => {
    s.present = Math.random() > 0.5;

    const row = table.rows[index];
    if (row) {
      row.cells[2].textContent =
        s.present ? "Present" : "Absent";
    }
  });
```

```
  console.timeEnd("DOM Update");
}

// ----------------------------
// VIRTUAL DOM
// ----------------------------
let oldVDOM = [];

function createVDOM(data) {
  return data.map(s => ({
    id: s.id,
    status: s.present ? "Present" : "Absent"
  }));
}

function diff(oldVDOM, newVDOM) {
  let changes = [];

  newVDOM.forEach((newNode, index) => {
    const oldNode = oldVDOM[index];

    if (!oldNode || oldNode.status !== newNode.status) {
      changes.push({ index, status: newNode.status });
    }
  });

  return changes;
}

function patchDOM(changes) {
  const table = document.getElementById("tableBody");

  changes.forEach(change => {
    const row = table.rows[change.index];
    if (row) {
      row.cells[2].textContent = change.status;

      // Highlight changed rows
      row.classList.add("updated");
    }
  });
}

// ----------------------------
// VDOM UPDATE (FAST)
// ----------------------------
function updateVDOM() {

  console.time("VDOM Update");

  students.forEach(s => {
    s.present = Math.random() > 0.5;
  });
```

```
  const newVDOM = createVDOM(students);

  if (oldVDOM.length === 0) {
    oldVDOM = createVDOM(students);
  }

  const changes = diff(oldVDOM, newVDOM);

  patchDOM(changes);

  oldVDOM = newVDOM;

  console.timeEnd("VDOM Update");
}
```

Output:



Student Attendance System

1 | Student 1 | Present

Add | Update

Update Using DOM | Update Using VDOM

ID	Name	Attendance	Action
1	Student 1	Present	Delete
2	Student 2	Absent	Delete
3	Student 3	Absent	Delete

Student Attendance System

1 | Student 1 | Present

Add | Update

Update Using DOM | Update Using VDOM

ID	Name	Attendance	Action
1	Student 1	Absent	Delete
2	Student 2	Absent	Delete
3	Student 3	Absent	Delete

Result: Thus, in the above task successfully executed without errors in VS Code.

Viva Questions & Answers

Q1: What is the DOM and why is direct manipulation slow?

The DOM is a live tree of objects representing the HTML document. Direct manipulation is slow because every write to the DOM can trigger reflow (layout recalculation) and repaint (pixel redrawing) in the browser's rendering pipeline. When many elements are updated in a loop, these operations accumulate and consume significant processing time.

Q2: What is the Virtual DOM and how does it improve performance?

The Virtual DOM is a lightweight JavaScript object tree that describes the desired UI state. It improves performance by computing the difference (diff) between the old and new state in pure JavaScript — which is fast — and then applying only the changed elements to the real DOM. This reduces the number of DOM mutations, reflows, and repaints.

Q3: What is reflow? What triggers it?

Reflow is the browser's process of recalculating the position and dimensions of all affected elements after a DOM change. It is triggered by any change that might affect element geometry: adding/removing nodes, changing text content, modifying font size, or reading layout properties (like offsetWidth) after a write operation.

Q4: What is the difference between reflow and repaint?

Reflow recalculates the geometry (position and size) of elements and is more expensive. Repaint redraws the visual appearance of elements (color, background, borders) without changing their geometry. Reflow always triggers repaint, but repaint does not always require reflow.

Q5: What does the diff algorithm do in this experiment?

The diff algorithm compares each node in the new Virtual DOM against the corresponding node in the old Virtual DOM. If a node's text content has changed, it records that node's index and new text in a changeset. If the texts are identical, the node is skipped. The resulting changeset contains only the nodes that need real DOM updates.

Q6: Why is console.time() used instead of Date.now()?

console.time() uses the high-resolution Performance API internally, providing sub-millisecond precision (up to microseconds). Date.now() returns whole millisecond values, which is not precise enough for operations that complete in under 10ms. console.time() also produces automatically labelled console output, making it convenient for comparison measurements.

Q7: How does this experiment relate to React?

React uses an identical conceptual approach. It maintains a Virtual DOM in memory, creates a new Virtual DOM tree on every state change, diffs the new tree against the previous tree using an O(n) algorithm called Fiber reconciliation, and commits only the changed nodes to the real DOM. The experiment demonstrates the foundational engineering insight that motivated React's creation.